

TD 5 : Initiation au langage PL/pgSQL

Sommaire

<u>Introduction</u>	1
<u>Exercice:1</u>	2
<u>Exercice:2</u>	2
<u>Exercice:3</u>	2
<u>Exercice:4</u>	2
<u>Exercice :5</u>	2
<u>Exercice:6</u>	2
<u>Exercice :7</u>	2
<u>Conclusion</u>	3

Introduction

Dans ce TP, nous allons découvrir le langage PL/pgSQL et apprendre à créer des fonctions dans PostgreSQL. Nous allons utiliser différentes structures de contrôle, des boucles et des fonctions prédéfinies afin de manipuler des chaînes de caractères, des dates et des données provenant d'une base de données.

Exercice:1

Écrire la fonction `calculer_longueur_max()` qui calcule la longueur de deux chaînes de caractères fournies en argument et qui retourne la longueur la plus longue. • Arguments en entrée : 2 chaînes de caractères. • Argument en sortie : longueur maximale des deux chaînes de caractères.

The screenshot shows a PostgreSQL IDE interface with the following components:

- Top Bar:** Dashboard, Properties, SQL, Statistics, Dependencies, Dependents, Processes, postgres/postgres..., hhb_direct_v1.sql*
- Toolbar:** Includes icons for file operations, query execution, and settings. A dropdown menu shows "No limit".
- Query Editor:** Contains the following SQL code:

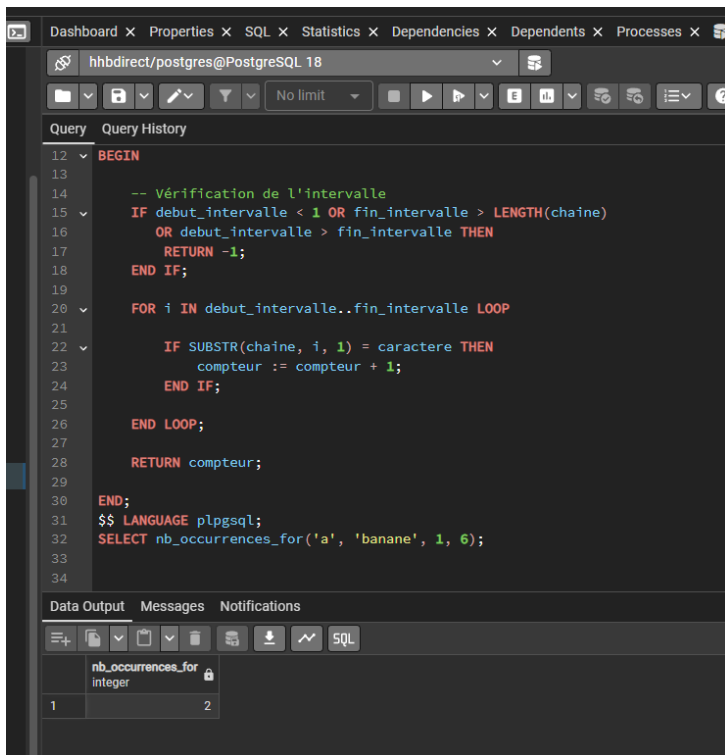

```

1 CREATE OR REPLACE FUNCTION calculer_longueur_max(
2     chaîne1 VARCHAR,
3     chaîne2 VARCHAR
4 )
5 RETURNS INTEGER
6 AS $$
7 DECLARE
8     longueur1 INTEGER;
9     longueur2 INTEGER;
10 BEGIN
11     longueur1 := LENGTH(chaîne1);
12     longueur2 := LENGTH(chaîne2);
13
14     IF longueur1 > longueur2 THEN
15         RETURN longueur1;
16     ELSE
17         RETURN longueur2;
18     END IF;
19 END;
20 $$ LANGUAGE plpgsql;
21 SELECT calculer_longueur_max('Bonjour', 'Salut');
22
23
      
```
- Data Output:** Shows the result of the function call:

calculer_longueur_max
integer
1

Exercice:2

1ère version en utilisant la boucle FOR



The screenshot shows a PostgreSQL IDE interface with a query editor. The query is a PL/pgSQL function named `nb_occurrences_for` that takes three parameters: `chaine`, `caractere`, and `debut_intervalle`. It uses a `FOR` loop to iterate from `debut_intervalle` to `fin_intervalle` and counts the occurrences of `caractere` in `chaine`. The function returns the count.

```

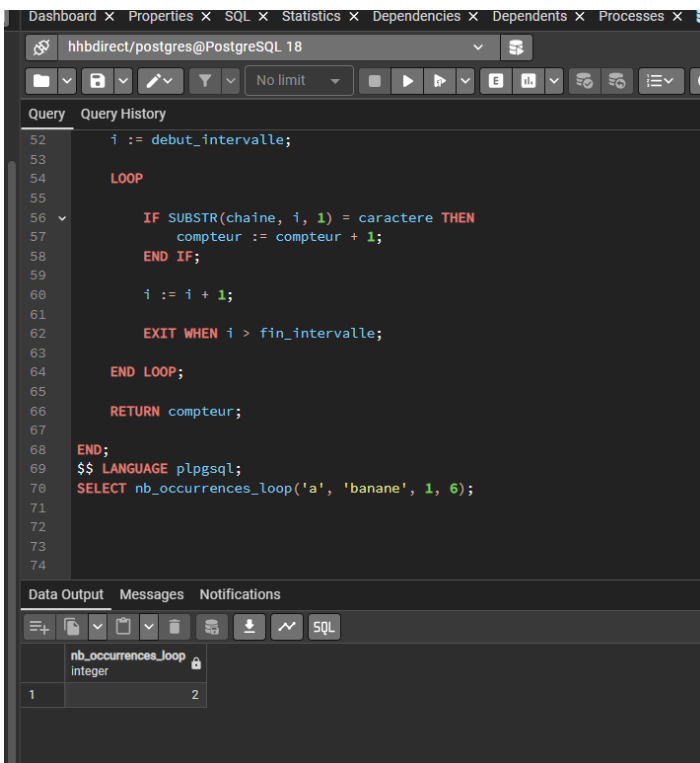
12 BEGIN
13
14 -- Vérification de l'intervalle
15 IF debut_intervalle < 1 OR fin_intervalle > LENGTH(chaine)
16 OR debut_intervalle > fin_intervalle THEN
17     RETURN -1;
18 END IF;
19
20 FOR i IN debut_intervalle..fin_intervalle LOOP
21
22     IF SUBSTR(chaine, i, 1) = caractere THEN
23         compteur := compteur + 1;
24     END IF;
25
26 END LOOP;
27
28 RETURN compteur;
29
30 END;
31 $$ LANGUAGE plpgsql;
32 SELECT nb_occurrences_for('a', 'banane', 1, 6);
33
34

```

The Data Output tab shows the result of the query:

nb_occurrences_for	integer
1	2

2ème version avec la boucle LOOP/EXIT



The screenshot shows a PostgreSQL IDE interface with a query editor. The query is a PL/pgSQL function named `nb_occurrences_loop` that takes three parameters: `chaine`, `caractere`, and `debut_intervalle`. It uses a `LOOP` with `EXIT WHEN` to iterate from `debut_intervalle` to `fin_intervalle` and counts the occurrences of `caractere` in `chaine`. The function returns the count.

```

52 i := debut_intervalle;
53
54 LOOP
55
56     IF SUBSTR(chaine, i, 1) = caractere THEN
57         compteur := compteur + 1;
58     END IF;
59
60     i := i + 1;
61
62     EXIT WHEN i > fin_intervalle;
63
64 END LOOP;
65
66 RETURN compteur;
67
68 END;
69 $$ LANGUAGE plpgsql;
70 SELECT nb_occurrences_loop('a', 'banane', 1, 6);
71
72
73
74

```

The Data Output tab shows the result of the query:

nb_occurrences_loop	integer
1	2

3ème version en utilisant la boucle WHILE

```

109         RETURN -1;
110     END IF;
111
112     i := debut_intervalle;
113
114     WHILE i <= fin_intervalle LOOP
115
116         IF SUBSTR(chaine, i, 1) = caractere THEN
117             compteur := compteur + 1;
118         END IF;
119
120         i := i + 1;
121
122     END LOOP;
123
124     RETURN compteur;
125
126 END;
127 $$ LANGUAGE plpgsql;
128
129 -- Test de la fonction WHILE
130 SELECT nb_occurrences_while('a', 'banane', 1, 6);
131

```

Data Output

nb_occurrences_while	integer
1	2

Exercice:3

Écrire la fonction getNbJoursParMois() qui calcule le nombre de jours dans un mois pour une date fournie en paramètre. • Argument en entrée : une date (sous format SQL). • Argument en sortie : nombre de jours du mois de la date

```

129
130 -- Test de la fonction WHILE
131 SELECT nb_occurrences_while('a', 'banane', 1, 6);
132
133 CREATE OR REPLACE FUNCTION getNbJoursParMois(
134     date_param DATE
135 )
136 RETURNS INTEGER
137 AS $$
138 BEGIN
139
140     RETURN EXTRACT(
141         DAY FROM (
142             DATE_TRUNC('month', date_param)
143             + INTERVAL '1 month'
144             - INTERVAL '1 day'
145         )
146     );
147
148 END;
149 $$ LANGUAGE plpgsql;
150 SELECT getNbJoursParMois('2025-02-15');
151

```

Data Output

getNbJoursParMois	integer
1	28

Exercice:4

Écrire la fonction `dateSqlToDatefr()` qui vous permet de convertir la date fournie en paramètre au format SQL en une date au format JJ/MM/AA. • Argument en entrée : une date au format AAAA-MM-JJ • Argument en sortie : une date au format JJ/MM/AA.

```

141         DAY FROM (
142             DATE_TRUNC('month', date_param)
143             + INTERVAL '1 month'
144             - INTERVAL '1 day'
145         )
146     );
147
148 END;
149 $$ LANGUAGE plpgsql;
150 SELECT getNbJoursParMois('2025-02-15');
151 CREATE OR REPLACE FUNCTION dateSqlToDatefr(
152     date_param DATE
153 )
154 RETURNS VARCHAR
155 AS $$
156 BEGIN
157     RETURN TO_CHAR(date_param, 'DD/MM/YY');
158
159 END;
160
161 $$ LANGUAGE plpgsql;
162 SELECT dateSqlToDatefr('2025-09-18');
163

```

Data Output

dateSqlToDatefr
18/09/25

Exercice :5

Écrire la fonction `getNomJour()` qui vous permet de retourner le nom du jour de la semaine correspondant à la date fournie en paramètre. • Argument en entrée : une date au format SQL • Argument en sortie : le nom du jour de la date. Contrainte : utiliser la fonction `extract()`.

```

181     WHEN 2 THEN
182         RETURN 'Mardi';
183
184     WHEN 3 THEN
185         RETURN 'Mercredi';
186
187     WHEN 4 THEN
188         RETURN 'Jeudi';
189
190     WHEN 5 THEN
191         RETURN 'Vendredi';
192
193     WHEN 6 THEN
194         RETURN 'Samedi';
195     END CASE;
196
197 END;
198 $$ LANGUAGE plpgsql;
199 SELECT getNomJour('2025-09-18');
200
201
202
203

```

Data Output

getNomJour
Jeudi

Exercice:6

Contexte : HHB-Direct (Ressource : e-sio/slam3/hhb_direct_v1.sql) Écrire une fonction qui retourne le nombre de clients débiteurs. Contrainte : utiliser la fonction `date_part()` de PostgreSQL. Écrire une fonction qui retourne le nombre de clients habitant dans une ville. Le nom de la ville est un paramètre de la fonction.

The screenshot shows a PostgreSQL IDE window titled 'hhbdirect/postgres@PostgreSQL 18'. The 'Query' tab is active, displaying SQL code for creating a function `nbClientsDebiteurs`. The code includes a `SELECT` statement to count distinct clients from the `posseder` table, joined with the `compte` table, where the balance is less than 0 and the account is opened in the year 2023. The function is created using `CREATE OR REPLACE FUNCTION` and `LANGUAGE plpgsql`. The 'Data Output' tab shows the result of the function call, which is 1.

```

211 ORDER BY ordinal_position;
212 SELECT *
213 FROM compte;
214 CREATE OR REPLACE FUNCTION nbClientsDebiteurs()
215 RETURNS INTEGER
216 AS $$
217 DECLARE
218     nombre INTEGER;
219 BEGIN
220
221     SELECT COUNT(DISTINCT p.num_client)
222     INTO nombre
223     FROM posseder p
224     JOIN compte c ON p.num_compte = c.num_compte
225     WHERE c.solde < 0
226           AND DATE_PART('year', c.date_ouvrir) = 2023;
227
228     RETURN nombre;
229
230 END;
231 $$ LANGUAGE plpgsql;
232 SELECT nbClientsDebiteurs();
233

```

nbclientsdebituers integer	
1	1

The screenshot shows a PostgreSQL IDE window titled 'hhbdirect/postgres@PostgreSQL 18'. The 'Query' tab is active, displaying SQL code for creating a function `nbClientsVille`. The code includes a `SELECT` statement to count all clients from the `client` table where the address matches the provided city. The function is created using `CREATE OR REPLACE FUNCTION` and `LANGUAGE plpgsql`. The 'Data Output' tab shows the result of the function call for the city 'LANNION', which is 2.

```

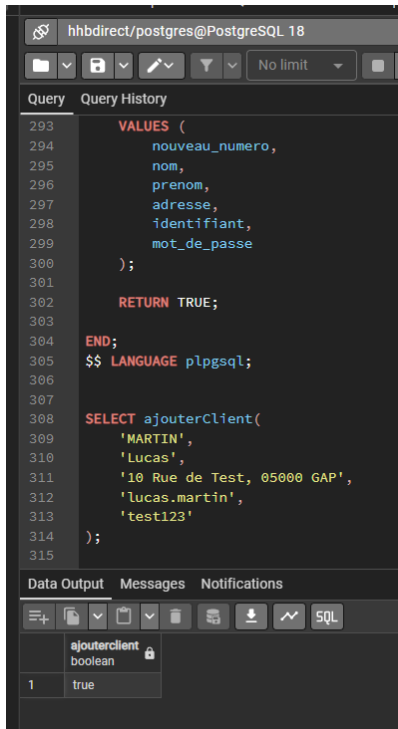
233 SELECT num_client, nom_client, prenom_client,
234 FROM client;
235 CREATE OR REPLACE FUNCTION nbClientsVille(
236     ville VARCHAR
237 )
238 RETURNS INTEGER
239 AS $$
240 DECLARE
241     nombre INTEGER;
242 BEGIN
243
244     SELECT COUNT(*)
245     INTO nombre
246     FROM client
247     WHERE adresse_client LIKE '%' || ville;
248
249     RETURN nombre;
250
251 END;
252 $$ LANGUAGE plpgsql;
253 SELECT nbClientsVille('LANNION');
254

```

nbclientsville integer	
1	2

Exercice :7

Écrire une fonction qui permet d'insérer un tuple dans la table client. Les valeurs des colonnes seront fournies en argument à la fonction sauf pour le numéro du client. Le numéro du client devra être calculé dans la fonction. La fonction récupère le dernier numéro et l'incrémente pour créer le nouveau numéro du client à enregistrer. Une information devra indiquer si le tuple a été bien enregistré ou pas.



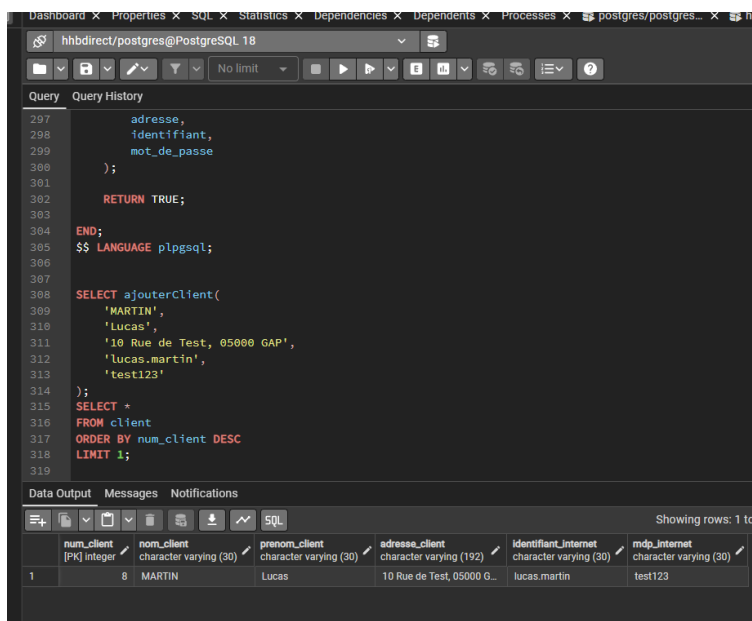
```

293 VALUES (
294     nouveau_numero,
295     nom,
296     prenom,
297     adresse,
298     identifiant,
299     mot_de_passe
300 );
301
302 RETURN TRUE;
303
304 END;
305 $$ LANGUAGE plpgsql;
306
307
308 SELECT ajouterClient(
309     'MARTIN',
310     'Lucas',
311     '10 Rue de Test, 05000 GAP',
312     'lucas.martin',
313     'test123'
314 );
315

```

Data Output

ajouterClient
boolean
1 true



```

297     adresse,
298     identifiant,
299     mot_de_passe
300 );
301
302 RETURN TRUE;
303
304 END;
305 $$ LANGUAGE plpgsql;
306
307
308 SELECT ajouterClient(
309     'MARTIN',
310     'Lucas',
311     '10 Rue de Test, 05000 GAP',
312     'lucas.martin',
313     'test123'
314 );
315
316 SELECT *
317 FROM client
318 ORDER BY num_client DESC
319 LIMIT 1;

```

Data Output

num_client	nom_client	prenom_client	adresse_client	identifiant_client	mdp_client
8	MARTIN	Lucas	10 Rue de Test, 05000 G...	lucas.martin	test123

Conclusion

Ce TP nous a permis de mettre en pratique le langage PL/pgSQL en créant différentes fonctions. Nous avons appris à utiliser des boucles, des conditions et des fonctions PostgreSQL pour effectuer des traitements sur les données.